

课程编号 1503020001-01

题目类型 实验 4

得分	教师签名	批改日期
	冯禹洪	

深圳大学实验报告

课程名称： 系统编程

实验项目名称： 文件控制

学院： 计算机与软件学院

专业： 计算机科学与技术（创新班）

指导教师： 冯禹洪

报告人： 李文俊 学号： 2023150001 班级： 高性能

实验时间： 2026. 6. 22

实验报告提交时间： 2026. 6. 22

教务处制

一、实验目标：

1. 熟悉文件控制相关的 Linux 操作系统的系统调用
2. 了解并掌握相关系统调用的应用程序编写

二、实验环境与工件

1. Taishan 服务器，openEuler 操作系统

三、实验内容与步骤

- 1、请阅读、编写并调试附件中的 `reverse.c` 程序, 使其满足以下用法和功能。请完成以下工作：1) 解释每个过程函数的功能；2) 在自己的姓名学号账号下运行并截图运行结果, 如果程序有错, 请截图注明修正代码及其原因(40 分)。

用法： `reverse -c [filename]`

作用：将从文件 `filename` (或标准输入) 读入的每一行翻转并打印到标准输出； 如果用了 `-c` 选项, `reverse` 也讲每行的字符翻转。

以下是 `reverse` 的用法与用例。

```
$ cc reverse.c -o reverse
$ cat test
```

```
Christmas is coming.
The days that grow shorter.
Remind me of seasons I knew in the past.
```

```
$ reverse test
```

```
Remind me of seasons I knew in the past.
The days that grow shorter.
Christmas is coming.
```

```
$ reverse -c test
```

```
.tsap eht ni wenk I snosaes fo em dnimeR
,retrohs worg taht syad ehT
,gnimoc si samtshirhC
```

```
$ cat test | reverse
```

```
Remind me of seasons I knew in the past.
The days that grow shorter,
Christmas is coming,
```

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

- 2、请阅读、编写并调试附件中的 monitor.c 程序,使其满足以下用法和功能。请完成以下工作:1) 解释每个过程函数的功能;2) 在自己的姓名学号账号下运行并截图运行结果,如果程序有错,请截图注明修正代码及其原因(40 分)。

用法: monitor [-t delay] [-l count] {filename}+

作用: 每 delay 秒扫描所有指定的文件并将从上次扫描以来更新过的文件的信息打印出来。如果文件名 filename 是一个目录,该目录中所有的文件都将被扫描。

以下是环境配置和 monitor 的用法与用例。

```
$mkdir filecontrol
$ ls                               // 查看 filecontrol 目录.
monitor.c      monitor      tmp/
$ls ./tmp      //查看 “tmp” 目录.
b
$monitor tmp myFile.txt >& monitor.out & //开始监控.
[1] 12841
$cat > ./tmp/a      //在./tmp 下创建一个文件.
hi there
ctrl-D
$cat > myFile.txt    //创建 “myFile.txt” .
hi there
ctrl-D
$rm ./tmp/a          //删除 “./tmp/a” .
$jobs                //查看正在运行的作业.
[1] + Running monitor tmp myFile.txt >& monitor.out
$kill %1              //kill “monitor” 作业.
[1] Terminated monitor tmp myFile.txt >& monitor.out
$cat monitor.out      //查看输入文件.
ADDED tmp/b size 9 bytes, mod. time = Sun Jan 18 00:38:55 2017
ADDED tmp/a size 9 bytes, mod. Time = Fri Feb 13 18:51:09 2017
ADDED myFile.txt size 9 bytes, mod. Time = Fri Feb 13 18:51:21
2017
CHANGED myFile.txt size 18 bytes, mod. Time = Fri Feb 13
18:51:49 2017
DELETED tmp/a
$
```

- 3、有四个线程 1、2、3、4. 线程 1 的功能就是输 1, 线程 2 的功能就是输出 2, 以此类推……现在有四个文件 ABCD. 初始都为空。现要让四个文件呈如下格式:

A:1 22 333 44441 22 333 4444...

B: 22 333 4444 1 22 333 4444 1...

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

C: 333 4444 1 22 333 4444 1 22...

D: 44441 22 333 44441 22 333...

请设计程序，附上代码截图及运行结果截图。（Google 面试题改编）（20分）

四、实验结果

1. 题 1：reverse 程序分析与修改

1.1 各过程函数的功能

reverse 的整体思路是“两遍扫描”：第一遍只记录每一行在文件中的起始偏移量，第二遍再按相反的行序依次定位、读取并输出。各过程函数的功能如下：

1. **main**：依次调用 `parseCommandLine`、`pass1`、`pass2`，是整个两遍扫描流程的入口。
2. **parseCommandLine**：遍历命令行参数，以“-”开头的交给 `processOptions` 处理，否则当作输入文件名；若文件名多于一个则报用法错误。最后据是否给出文件名设置 `standardInput` 标志。
3. **processOptions**：逐个解析形如“-c”的选项字符，遇到 `c` 置位 `charOption`（表示需要翻转每行字符），其余选项报错。
4. **usageError**：向标准错误打印用法提示并以失败状态退出。
5. **pass1**：第一遍扫描。从标准输入读入时先把数据复制到一个临时文件，以便第二遍能够用 `lseek` 随机定位；同时调用 `trackLines` 记录每行起始偏移。
6. **trackLines**：扫描缓冲区中的每个字符，维护当前文件偏移 `fileOffset`，每遇到一个换行符就把下一行的起始偏移记入 `lineStart` 数组。
7. **pass2**：第二遍扫描。按行号从大到小调用 `processLine`，从而实现“整体行序翻转”，最后关闭文件并在使用临时文件时将其删除。
8. **processLine**：用 `lseek` 定位到第 `i` 行起始处，按该行长度 `read` 出整行；若指定了 `-c` 选项则调用 `reverseLine` 翻转字符，再写到标准输出。
9. **reverseLine**：对一行字符做首尾对调的原地翻转，并保留行尾换行符的位置。
10. **fatalError**：用 `perror` 报告系统调用错误并退出。

1.2 程序存在的问题与修改

附件中的 `reverse.c` 直接编译无法通过，逐项排查后发现并修正了以下问题：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

1. **源文件含非法字符（最主要的错误）**：原文件中所有的单/双引号都变成了乱码字节（如“ - ” “ c ”），导致 gcc 报“source file is not valid UTF-8”。修改：把所有有字符常量与字符串常量改回标准 ASCII 引号，如 'c'、'\n'、".rev.%d"、"Usage: reverse -c [filename]\n"。
2. **变量名拼写错误**：pass1 中将 standardInput 误写为 standarInput，属未声明标识符。修改：更正为 standardInput。
3. **缺少头文件与函数原型**：read/write/open/lseek/close/unlink/getpid 来自 <unistd.h>，原文件未包含，加之 K&R 旧式函数定义在 C99 以后被视为“隐式 int / 隐式声明”错误。修改：补上 #include <unistd.h>，并将所有函数改写为带原型的 ANSI 风格、在文件开头集中声明。
4. **行尾无换行的末行会丢失**：原 pass1 仅靠换行符计数，若文件最后一行不以 \n 结尾，pass2 的循环 i=lineCount-1...0 不会覆盖这一行，导致末行被丢弃。修改：在 pass1 结尾判断是否存在未终结的末行并补登一条边界。关键代码如下：

```
/* 处理结尾没有换行符的最后一行，避免其被丢弃 */
if (fileOffset > lineStart[lineCount])
    ++lineCount;

lineStart[lineCount] = fileOffset; /* 末行的结束边界 */
```

图 1 reverse 末行无换行的修正核心代码

原始程序的编译报错如图 2 所示。

```
liwenjun2023150001@euler-container-30001: ~/exp4
[liwenjun2023150001@euler-container-30001 exp4]$ gcc -Wall -o reverse reverse.c # 直接编译附件原文件
reverse_orig.c:44:42: error: stray '\241' in program
reverse_orig.c:44:43: error: stray '\256' in program
reverse_orig.c:44:45: error: stray '\241' in program
reverse_orig.c:44:46: error: stray '\257' in program
reverse_orig.c:44:48: error: expected expression before ')' token
reverse_orig.c:63:16: error: stray '\241' in program
reverse_orig.c:63:17: error: stray '\256' in program
reverse_orig.c:63:19: error: stray '\241' in program
... (编译失败：非法 UTF-8 引号、隐式 int、未声明函数等)
```

图 2 reverse.c 原始程序的编译报错（非法 UTF-8 引号、K&R 隐式声明等）

1.3 运行结果

修正后用 gcc -Wall 编译通过。如图 3 所示：不加 -c 时只翻转“行的顺序”，加 -c 时在翻转行序的同时翻转每行的字符；对结尾无换行符的输入 (one/two/three) 末行 three 也能正确输出，验证了 1.2 的末行修正。

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
- 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```
liwenjun2023150001@euler-container-30001: ~/exp4
[liwenjun2023150001@euler-container-30001 exp4]$ gcc -Wall -o reverse reverse.c
[liwenjun2023150001@euler-container-30001 exp4]$ printf 'hello world\nfoo bar\nabcde\n' > t1.txt
[liwenjun2023150001@euler-container-30001 exp4]$ cat t1.txt
hello world
foo bar
abcde
[liwenjun2023150001@euler-container-30001 exp4]$ ./reverse t1.txt
abcde
foo bar
hello world
[liwenjun2023150001@euler-container-30001 exp4]$ ./reverse -c t1.txt
edcba
rab oof
dlrow olleh
[liwenjun2023150001@euler-container-30001 exp4]$ printf 'one\ntwo\nthree' | ./reverse
threetwo
one
```

图 3 reverse 运行结果（行序翻转 / -c 字符翻转 / 末行无换行）

2. 题 2: monitor 程序分析与修改

2.1 各过程函数的功能

monitor 周期性地对命令行给出的文件/目录调用 stat，并把自上次扫描以来“新增 / 改动 / 删除”的文件打印出来。各过程函数的功能如下：

1. **main**: 调用 `parseCommandLine` 解析参数后进入 `monitorLoop` 主循环。
2. **parseCommandLine**: 把以 “-” 开头的参数交给 `processOptions`，其余作为待监控的文件名收集到 `fileNames` 数组；无文件名则报用法错误。
3. **processOptions**: 解析 `-t`（扫描间隔秒数）和 `-l`（循环次数）两个选项，数字通过 `getNumber` 取得。
4. **getNumber**: 从选项字符后面连续读取数字字符并转换成整数（如 `-t5` 中的 5）。
5. **monitorLoop**: 主循环。每轮调用 `monitorFiles` 扫描一遍，刷新输出后 `sleep(delayTime)`，按 `loopCount` 控制次数（-1 表示无限）。
6. **monitorFiles**: 对每个命令行文件调用 `monitorFile`；一轮结束后遍历状态数组，凡是“上轮在、本轮不在”的文件打印 DELETED，并把 `thisCycle` 滚动为 `lastCycle`。
7. **monitorFile**: 对单个路径 `stat`，若是目录则递归 `processDirectory`，若是普通/字符/块设备文件则交给 `updateStat`。
8. **processDirectory**: 打开目录并遍历其中每个条目（跳过 `.` 和 `..`），拼出完整路径后递归调用 `monitorFile`。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

9. **updateStat**: 在状态数组中查找该文件, 没有就 addEntry 新增、有就 updateEntry 更新, 并把 thisCycle 置真表示本轮出现过。
10. **findEntry / nextFree**: 分别在状态数组中按文件名查找已有条目、寻找一个空闲槽位。
11. **addEntry**: 占用空槽, 保存文件名与 stat 信息, 打印 ADDED。
12. **updateEntry**: 比较修改时间 st_mtime, 若变化则更新并打印 CHANGED。
13. **printEntry / printStat**: 打印文件名、大小与修改时间 (asctime 格式)。
14. **fatalError**: 用 perror 报告错误并退出。

2.2 程序存在的问题与修改

monitor.c 同样无法直接编译, 且存在若干逻辑笔误, 逐项修正如下:

1. **源文件含非法字符**: 与 reverse.c 相同, 所有引号均为乱码字节, 导致非法 UTF-8 报错。修改: 全部改回标准 ASCII 引号。
2. **大小写笔误 argv[I]**: parseCommandLine 中把循环变量 i 误写成大写 I (未声明)。修改: 改回 argv[i]。
3. **函数自我递归笔误**: monitorFiles (复数) 内部本应调用 monitorFile (单数) 逐个处理文件, 原文件误写成调用自身 monitorFiles, 会导致无限递归。修改: 改为 monitorFile(fileName[i])。
4. **数组名笔误 stat[i]**: 检测删除时把状态数组 stats 误写成 stat。修改: 改回 stats[i].thisCycle。
5. **宏名拼写 S_ISBLCK**: 判断块设备的宏应为 S_ISBLK, 原文件多写了一个 C。修改: 改为 S_ISBLK。
6. **缺少分号**: processDirectory 中拼接路径的 sprintf 语句末尾漏写分号。修改: 补上分号。
7. **函数名拼写 ascTime**: 标准库函数应为 asctime (全小写)。修改: 改为 asctime。
8. **getdents 不可移植**: 原文件用裸系统调用 getdents 配合 lseek(d_off) 读目录, 在 openEuler/现代 glibc 上既不使用又易出错。修改: 改用标准库的 opendir/readdir/closedir 遍历目录, 并补齐 <stdlib.h>、<unistd.h>、<dirent.h> 等头文件。关键代码如下:

```
DIR *dir = opendir(dirName);
if (dir == NULL) fatalError();
while ((entry = readdir(dir)) != NULL) {
    if (strcmp(entry->d_name, ".") && strcmp(entry->d_name, "..")) {
        sprintf(fileName, "%s/%s", dirName, entry->d_name);
```

- 注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

        monitorFile(fileName);          /* 递归处理 */
    }
}

closedir(dir);

```

图 4 monitor 目录遍历改用 opendir / readdir 的核心代码

原始程序的编译报错如图 5 所示。

```

liwenjun2023150001@euler-container-30001: ~/exp4
[liwenjun2023150001@euler-container-30001 exp4]$ gcc -Wall -o monitor monitor.c # 直接编译附件原文件
monitor_orig.c:5:10: fatal error: sys/dirent.h: No such file or directory
 5 | #include <sys/dirent.h> /* For getdents */
   |          ^
[liwenjun2023150001@euler-container-30001 exp4]$ # 暂将 <sys/dirent.h> 换成 <dirent.h> 以暴露其余错误
/tmp/mo.c:46:19: error: stray '\241' in program
/tmp/mo.c:46:20: error: stray '\256' in program
/tmp/mo.c:46:22: error: stray '\241' in program
/tmp/mo.c:46:23: error: stray '\257' in program
/tmp/mo.c:46:21: error: 't' undeclared (first use in this function)
/tmp/mo.c:49:19: error: stray '\241' in program
/tmp/mo.c:49:20: error: stray '\256' in program
... (编译失败: 非法字符、argv[I]/stat[i] 笔误、S_ISBLK、ascTime 等)

```

图 5 monitor.c 原始程序的编译报错（argv[I]、非法引号、隐式声明等）

2.3 运行结果

修正后按用例验证：先在 tmp 目录放入文件 b，启动 monitor 后台监控 tmp 与 myFile.txt，随后依次创建 tmp/a、创建并追加 myFile.txt、删除 tmp/a。如图 6 所示，monitor 依次打印出 ADDED tmp/b、ADDED tmp/a、ADDED myFile.txt、CHANGED myFile.txt、DELETED tmp/a，与题目用例完全一致。（选项采用紧贴写法 -t1 -l6，与 getNumber 从选项字符后读数字的实现相符。）

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
 2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。


```
liwenjun2023150001@euler-container-30001: ~/exp4
[liwenjun2023150001@euler-container-30001 exp4]$ ls ; ls tmp
A
B
C
D
GoogleInterview
GoogleInterview.c
monitor
monitor.c
monitor_orig.c
reverse
reverse.c
reverse_orig.c
t1.txt
tmp
---tmp---
b
[liwenjun2023150001@euler-container-30001 exp4]$ ./monitor -t1 -l6 tmp myFile.txt >& monitor.out &
[1] (后台启动: 随后依次创建 tmp/a、创建并追加 myFile.txt、删除 tmp/a)
[liwenjun2023150001@euler-container-30001 exp4]$ cat monitor.out
ADDED tmp/b size 7 bytes, mod. time = Tue Jun  2 16:13:07 2026
ADDED tmp/a size 9 bytes, mod. time = Tue Jun  2 16:13:08 2026
ADDED myFile.txt size 9 bytes, mod. time = Tue Jun  2 16:13:09 2026
CHANGED myFile.txt size 21 bytes, mod. time = Tue Jun  2 16:13:10 2026
DELETED tmp/a
```

图 6 monitor 运行结果 (ADDED / CHANGED / DELETED)

3. 题 3：四个线程错位写入 A/B/C/D 四个文件

3.1 设计思路

线程 t ($t=1,2,3,4$) 每次写出 t 个字符 t (即 "1" "22" "333" "4444")，四个文件要求呈错位轮转：A 从 1 开始、B 从 22 开始、C 从 333 开始、D 从 4444 开始。要同时满足“写出顺序固定为 1→2→3→4 轮流”与“四个文件各自错位”，关键有两点：

1. **严格顺序**：用一把互斥锁加一个条件变量、配合全局计数器 `counter` 实现令牌轮转——只有当 $\text{counter} \% 4 == t-1$ 时线程 t 才被允许写入，写完 $\text{counter}++$ 再 broadcast 唤醒下一个线程；wait 用 while 包裹以防虚假唤醒。
2. **错位路由**：把连续四次写入看作一“轮” $\text{round} = \text{counter} / 4$ 。令 token 为 t 的写入落到文件下标 $f = (t-1-\text{round}) \% 4$ ($0=A, 1=B, 2=C, 3=D$)，即可让每一轮恰好给四个文件各写一个 token，且各文件内部正好实现题目要求的错位轮转。

3.2 关键代码

线程函数 worker 的核心是“等到自己的令牌→计算本轮目标文件→写入→交棒”：

```
while (counter % NTHREAD != t - 1)      /* 没轮到自己就等待 */
    pthread_cond_wait(&cond, &mutex);

f = ((t - 1 - round) % NTHREAD + NTHREAD) % NTHREAD; /* 本轮目标文件 */
```

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

```

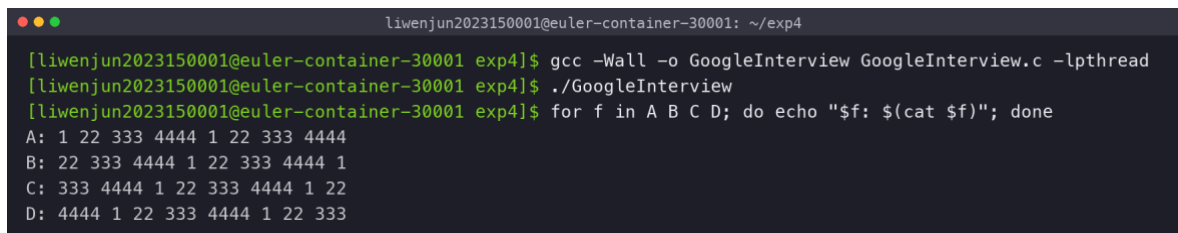
for (j = 0; j < t; j++) token[j] = '0' + t;          /* 拼出 "tttt" */
token[t] = ' ';
write(fds[f], token, t + 1);                        /* 写入并以空格分隔 */
counter++;                                           /* 交棒给下一个线程 */
pthread_cond_broadcast(&cond);

```

图 7 四线程令牌轮转与错位路由核心代码

3.3 运行结果

编译运行后查看四个文件的内容，如图 8 所示，A/B/C/D 分别从 1、22、333、4444 开始错位轮转，与题目要求完全一致。



```

liwenjun2023150001@euler-container-30001: ~/exp4
[liwenjun2023150001@euler-container-30001 exp4]$ gcc -Wall -o GoogleInterview GoogleInterview.c -lpthread
[liwenjun2023150001@euler-container-30001 exp4]$ ./GoogleInterview
[liwenjun2023150001@euler-container-30001 exp4]$ for f in A B C D; do echo "$f: $(cat $f)"; done
A: 1 22 333 4444 1 22 333 4444
B: 22 333 4444 1 22 333 4444 1
C: 333 4444 1 22 333 4444 1 22
D: 4444 1 22 333 4444 1 22 333

```

图 8 GoogleInterview 运行结果（A/B/C/D 四个文件的错位轮转内容）

五、实验总结与体会

本次实验围绕 Linux 文件控制相关的系统调用，完整地经历了“读代码—找错误—改正—编译运行—验证”的过程。题 1 的 reverse 让我熟悉了 open/read/write/lseek/close/unlink 这一组面向文件描述符的底层 I/O：它通过两遍扫描——第一遍用 lseek 配合偏移数组为每一行建立索引，第二遍再逆序定位输出——巧妙地用随机定位实现了整文件的行序翻转。

题 2 的 monitor 则把重点放在文件元数据与目录遍历上：通过 stat 读取 st_mtime、st_size 等信息，配合 lastCycle/thisCycle 两个标志位的滚动，就能判断文件的新增、改动与删除；我把原始程序里不可移植的 getdents 改写为标准的 opendir/readdir，使目录递归扫描在 openEuler 上稳定工作。这两个程序的共同特点是附件源码都被“引号乱码 + K&R 旧式语法 + 多处笔误”困扰，让我体会到：拿到一份不能编译的代码时，先按编译器报错从“非法字符、缺头文件、隐式声明”这些根因入手，再逐一排查 argv[i]、stat[i]、S_ISBLCK、ascTime 这类肉眼易漏的笔误，往往比急着改逻辑更有效。

题 3 的四线程错位写文件把同步与文件写入结合在一起。我用“互斥锁 + 条件变量 + 全局计数器”实现严格的令牌轮转，再用一个 $round = counter/4$ 、 $f = (t-1-round) \bmod 4$ 的取模公式把每一轮的写入精确路由到 A/B/C/D，既保证了 1→2→3→4 的输出顺序，又一次性满足了四个文件的错位要求。这让我更深刻地理解到：多线程程序的正确性来自“对共享状态的有序访问”，而把复杂的排布需求拆解成“顺序控制 + 下标映射”两层，往往能让实现既简单又可验证。

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

指导教师批阅意见：	
成绩评定：	
禹洪 日	指导教师签字：冯 2026 年 月
备注：	

注：1、报告内的项目或内容设置，可根据实际情况加以调整 and 补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。